12

Summarizing Data

In this chapter, you will learn what the SQL aggregate functions are and how to use them to summarize table data.

Using Aggregate Functions

It is often necessary to summarize data without actually retrieving it all, and MySQL provides special functions for this purpose. You can use these functions in MySQL queries to retrieve data for analysis and reporting purposes. These are some examples of this type of retrieval:

- Determining the number of rows in a table (or the number of rows that meet some condition or contain a specific value)
- Obtaining the sum of a group of rows in a table
- Finding the highest, lowest, and average values in a table column (either for all rows or for specific rows)

In each of these examples, you want a summary of the data in a table, not the actual data itself. Therefore, returning the actual table data would be a waste of time and processing resources (not to mention bandwidth). To repeat: All you really want is the summary information.

To facilitate this type of retrieval, MySQL features a set of aggregate functions, some of which are listed in Table 12.1. These functions enable you to perform all the types of retrieval just enumerated.

New Term

Aggregate Function A function that operates on a set of rows to calculate and return a single value.

Table 12.1 SQL Aggregate Functions

Function	Description
`Avg()`	Returns a column's average value
`Count()`	Returns the number of rows in a column
`Max()`	Returns a column's highest value
`Min()`	Returns a column's lowest value
`Sum()`	Returns the sum of a column's values

The use of each of these functions is explained in the following sections.

> **Note**
> **Standard Deviation** A series of standard deviation aggregate functions are also supported by MySQL, but they are not covered in this book.

The Avg() Function

`Avg()` is used to return the average value of a specific column by counting both the number of rows in the table and the sum of their values. `Avg()` can be used to return the average value of all columns or of specific columns or rows.

This first example uses `Avg()` to return the average price of all the products in the `products` table:

► Input

```
SELECT Avg(prod_price) AS avg_price
FROM products;
```

► Output

```
+-----------+
| avg_price |
+-----------+
| 16.133571 |
+-----------+
```

► Analysis

This `SELECT` statement returns a single value, `avg_price`, that contains the average price of all products in the `products` table. `avg_price` is an alias (refer to Chapter 10, "Creating Calculated Fields").

`Avg()` can also be used to determine the average value of specific columns or rows.

The following example returns the average price of products offered by a specific vendor:

Input

```
SELECT Avg(prod_price) AS avg_price
FROM products
WHERE vend_id = 1003;
```

Output

```
+-----------+
| avg_price |
+-----------+
| 13.212857 |
+-----------+
```

Analysis

This `SELECT` statement differs from the previous one only in that this one contains a `WHERE` clause. The `WHERE` clause filters to show only products with a `vend_id` of `1003`, and, therefore, the value returned in `avg_price` is the average of just that vendor's products.

Caution

Individual Columns Only You can use `Avg()` to determine the average of a specific numeric column, and that column name must be specified as the function parameter. To obtain the average value of multiple columns, you need to use multiple `Avg()` functions.

Note

`NULL` Values The `Avg()` function ignores column rows that contain `NULL` values.

The `Count()` Function

`Count()` does what it sounds like it does: It counts. Using `Count()`, you can determine the number of rows in a table or the number of rows that match a specific criterion.

You can use `Count()` in two ways:

- Use `Count(*)` to count the number of rows in a table, whether columns contain values or `NULL` values.
- Use `Count(column)` to count the number of rows that have values in a specific column, ignoring `NULL` values.

This example returns the total number of customers in the `customers` table:

Input

```
SELECT Count(*) AS num_cust
FROM customers;
```

Output

```
+----------+
| num_cust |
+----------+
|        5 |
+----------+
```

Analysis

In this example, `Count(*)` is used to count all rows, regardless of values. The count is returned in `num_cust`.

The following example counts just the customers with email addresses provided:

Input

```
SELECT Count(cust_email) AS num_cust
FROM customers;
```

Output

```
+----------+
| num_cust |
+----------+
|        3 |
+----------+
```

Analysis

This `SELECT` statement uses `Count(cust_email)` to count only rows with a value in the `cust_email` column. In this example, `cust_email` is `3` (meaning that only three of the five customers have email addresses listed).

Note

`NULL` Values The `Count()` function ignores column rows with `NULL` values in them if a column name is specified but not if the asterisk (`*`) is used.

The `Max()` Function

`Max()` returns the highest value in a specified column. `Max()` requires that the column name be specified, as shown here:

Input

```
SELECT Max(prod_price) AS max_price
FROM products;
```

Output

```
+-----------+
| max_price |
+-----------+
|     55.00 |
+-----------+
```

Analysis

Here `Max()` returns the price of the most expensive item in the `products` table.

Tip

Using `Max()` with Non-numeric Data Although `Max()` is usually used to find the highest numeric or date values, MySQL allows it to be used to return the highest value in any column—including textual columns. When used with textual data, `Max()` returns the row that would be the last row if the data were sorted by that column.

Note

`NULL` Values The `Max()` function ignores column rows with `NULL` values in them.

The `Min()` Function

`Min()` is exactly the opposite of `Max()`: It returns the lowest value in a specified column. Like `Max()`, `Min()` requires that the column name be specified, as shown here:

Input

```
SELECT Min(prod_price) AS min_price
FROM products;
```

Output

```
+-----------+
| min_price |
+-----------+
| 2.50      |
+-----------+
```

Analysis

Here `Min()` returns the price of the least expensive item in the `products` table.

Tip

Using `Min()` with Non-numeric Data As with the `Max()` function, MySQL allows `Min()` to be used to return the lowest value in any columns, including textual columns. When used with textual data, `Min()` returns the row that would be first if the data were sorted by that column.

> **Note**
> **NULL Values** The `Min()` function ignores column rows with `NULL` values in them.

The Sum() Function

`Sum()` is used to return the sum (total) of the values in a specific column. Here is an example to demonstrate this. The `orderitems` table contains the actual items in an order, and each item has an associated `quantity`. The total number of items ordered (the sum of all the `quantity` values) can be retrieved as follows:

Input

```
SELECT Sum(quantity) AS items_ordered
FROM orderitems
WHERE order_num = 20005;
```

Output

```
+---------------+
| items_ordered |
+---------------+
| 19            |
+---------------+
```

Analysis

The function `Sum(quantity)` returns the sum of all the item quantities in an order, and the `WHERE` clause ensures that just the right order items are included.

`Sum()` can also be used with the mathematical operators introduced in Chapter 10. In this next example, the total order amount is retrieved by totaling `item_price*quantity` for each item:

Input

```
SELECT SUM(item_price*quantity) AS total_price
FROM orderitems
WHERE order_num = 20005;
```

Output

```
+-------------+
| total_price |
+-------------+
|      149.87 |
+-------------+
```

Analysis

The function `Sum(item_price*quantity)` returns the sum of all the expanded prices in an order, and again the `WHERE` clause ensures that just the correct order items are included.

Tip

Performing Calculations on Multiple Columns All the aggregate functions can be used to perform calculations on multiple columns using the standard mathematical operators, as shown in this example.

Note

`NULL` Values The `Sum()` function ignores column rows with `NULL` values in them.

Aggregates on Distinct Values

The five aggregate functions can all be used in two ways:

- To perform calculations on all rows, specify the `ALL` argument or specify no argument at all (because `ALL` is the default behavior).
- To include only unique values, specify the `DISTINCT` argument.

Tip

`ALL` Is the Default The `ALL` argument need not be specified because it is the default behavior. If `DISTINCT` is not specified, `ALL` is assumed.

The following example uses the `Avg()` function to return the average product price offered by a specific vendor. It is the same `SELECT` statement used in the previous example, but here the `DISTINCT` argument is used so the average takes into account only unique prices:

Input

```
SELECT Avg(DISTINCT prod_price) AS avg_price
FROM products
WHERE vend_id = 1003;
```

Output

```
+-----------+
| avg_price |
+-----------+
| 15.998000 |
+-----------+
```

Analysis

As you can see, in this example, `avg_price` is higher when `DISTINCT` is used because there are multiple items with the same lower price. Excluding them raises the average price.

Caution

No DISTINCT with Wildcards DISTINCT can be used with Count() only if a column name is specified. DISTINCT cannot be used with Count(*), and so Count(DISTINCT *) is not allowed and generates an error. Similarly, DISTINCT must be used with a column name and not with a calculation or an expression.

Tip

Using DISTINCT with Min() and Max() Although you can technically use DISTINCT with Min() and Max(), there is actually no value in doing so. The minimum and maximum values in a column are the same whether only distinct values are included or not.

Combining Aggregate Functions

All the examples of aggregate functions used thus far have involved a single function. But actually, SELECT statements may contain as few or as many aggregate functions as needed. Look at this example:

Input

```
SELECT Count(*) AS num_items,
       Min(prod_price) AS price_min,
       Max(prod_price) AS price_max,
       Avg(prod_price) AS price_avg
FROM products;
```

Output

```
+-----------+-----------+-----------+-----------+
| num_items | price_min | price_max | price_avg |
+-----------+-----------+-----------+-----------+
|        14 |      2.50 |     55.00 | 16.133571 |
+-----------+-----------+-----------+-----------+
```

Analysis

Here a single SELECT statement performs four aggregate calculations in one step and returns four values (the number of items in the products table and the highest, lowest, and average product prices).

Tip

Naming Aliases When specifying the name of an alias to contain the results of an aggregate function, try to not use the name of an actual column in the table. Although there is nothing actually illegal about doing so, using unique names makes your SQL easier to understand and work with (and troubleshoot in the future).

Summary

Aggregate functions are used to summarize data. MySQL supports a range of aggregate functions, all of which can be used in multiple ways to return just the results you need. These functions are designed to be highly efficient, and they usually return results far more quickly than you could calculate them yourself within your own client application.

Challenges

1. Write a SQL statement to determine the total number of items sold (using the `quantity` column in `OrderItems`).
2. Modify the statement you just created to determine the total number of product item (`prod_item`) `BR01` sold.
3. This is a bit of a silly one, but imagine that a customer wants to buy one of every single item in the `products` table. Write a SQL statement to calculate the total price.
4. Write a SQL statement to determine the price (`prod_price`) of the most expensive item in the `Products` table that costs no more than 10. Name the calculated field `max_price`.

This page intentionally left blank